HTML, CSS & JS

Gestión de Datos en Entornos Inteligentes (XDEI)

HTML
*(**H**yper**T**ext **M**arkup **L**anguage)*

HTML, CSS & JS

HTML

- Permite definir la **estructura** de una página web
  - A nivel **semántico**, NO de formato

- **Estructura básica**
  - Cabecera **<head>**
  - Cuerpo **<body>**

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="utf-8" />
    <title>My test page</title>
  </head>
  <body>
    <p>This is my page</p>
  </body>
</html>
```

HTML
Estructura básica – Cabecera **<head>**

- Metadatos **<meta>**
  - *charset:* establece codificación de caracteres
- Título **<title>**
  - Texto que irá en el título de la ventana del navegador
- Estilo **<style>**
  - Define estilos CSS
- Enlace **<link>**: incluye elementos externos
  - Una hoja CSS
  - Un icono favorito
  - Una fuente (tipografía)
- Código ejecutable **<script>**: incluye un *script* JS

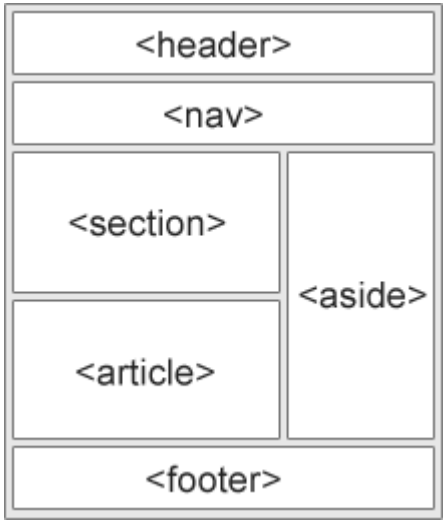
```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="utf-8" />
    <title>My test page</title>
    <style>p {    color: blue;  }</style>
    <link href="/static/example.css"
              rel="stylesheet" />
     <script src="/scripts/javascript.js">
     </script>
  </head>
  <body>
    <p>This is my page</p>
  </body>
```

HTML
Estructura básica – Cuerpo **<body>**

- Elementos de **contenido**
  - Encabezados <**h1**>, <**h2**>…
  - Párrafos <**p**>
  - Listas
    - Ordenadas <**ol**>
    - Sin orden <**ul**>
  - Tablas <**table**>
  - Imágenes <**img**>, audio <**audio**>, video <**video**>
  - Enlaces (anclajes) <**a**>
- Elementos de **resaltado semántico**
  - <**strong**>, <**em**>, <**cite**>, <**address**>, <**abbr**>…

HTML
Estructura básica – Cuerpo <body>

- Elementos de ***layout***
  - Encabezado <**header**> y pie <**footer**>
  - Barra de navegación <**nav**>
  - Sección <**section**>
  - Artículo <**article**>
  - Explicación al margen <**aside**>



- Elementos **contenedores** (divisiones)
  - De bloque <**div**>
  - En la misma línea <**span**>
- Elementos de **entrada de datos (formulario <form>)**
  - Etiqueta <**label**>
  - Entrada <**input**>

HTML
Atributos de un elemento

- Atributos: *lang*, *id*, *class*, *href* (`<a>`), *src* (`<img>`)…
- Atributos de **identificación**
  - Un único elemento (***id***)
  - Una clase de elementos (***class***)
- Con los atributos *id* y *class* podemos referenciar elementos del documento HTML desde una hoja CSS para darles formato
- Un elemento
  - **Solo** puede tener **un *id***
  - Puede pertenecer a **varias clases**: los estilos de clase se aplican en el orden que aparecen en la hoja CSS
- El atributo class es el pilar sobre el que se construyen los ***frameworks* CSS**
  - **Bootstrap** es el más conocido

HTML
Atributos *id* y *class*

- Ejemplo

HTML

```
<p class="help">Texto de ayuda</p>
<p class="help" id="help-item-1">Ayuda específica núm. 1</p>
<p class="help help-chatgpt">Ayuda proporcionada por ChatGPT</p>
```

CSS

```
.help {
     color: blue;
}
#help-item-1 {
     font-style: italic;
}
.help-chatgpt {
     font-variant: small-caps;
}
```

RESULTADO

Texto de ayuda

Ayuda específica núm. 1

AYUDA PROPORCIONADA POR CHATGPT

HTML
Formularios

- Ejemplo

First name:
John
Last name:
Doe

```
<form>
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe">
</form>
```

- Ejemplo

☐ I have a bike
☐ I have a car
☐ I have a boat

Submit

```
<form action="/action_page“ method="post" target="_blank">
  <input type="checkbox" id="vehicle1" name="vehicle1" value="Bike">
  <label for="vehicle1"> I have a bike</label><br>
  <input type="checkbox" id="vehicle2" name="vehicle2" value="Car">
  <label for="vehicle2"> I have a car</label><br>
  <input type="checkbox" id="vehicle3" name="vehicle3" value="Boat">
  <label for="vehicle3"> I have a boat</label><br><br>
  <input type="submit" value="Submit">
</form>
```

HTML
Formularios (II)

- Ejemplo *(Product)*

Id:

Name:

Size:

Price:

Submit

```
<form action="/products/create">
  <label for="id">Id:</label><br>
  <input type="text" id="id" name="id"><br>
  <label for="name">Name:</label><br>
  <input type="text" id="name" name="name"><br>
  <label for="size">Size:</label><br>
  <input type="text" id="size" name="size"><br>
  <label for="name">Price:</label><br>
  <input type="number" id="price" name="price"><br><br>
  <input type="submit" value="Submit">
</form>
```

HTML
Formularios – Procesado del servidor

- Misma URL para mostrar el formulario (petición GET) que para recibir los valores introducidos (petición POST)

```
@app.route("/products/create", methods=['GET', 'POST'])
def create_product():
    if request.method == 'POST':
        product = {"id": request.form["id"],
                   "type": "Product",
                   "name": {"type": "Text", "value": request.form["name"]},
                   "size": {"type": "Text", "value": request.form["size"]},
                   "price": {"type": "Integer", "value": int(request.form["price"])}}
        status = ngsiv2.create_entity(product)
        if status == 201:
            next = request.args.get('next', None)
            if next:
                return redirect(next)
            return redirect(url_for('display_products'))
    else:
        return render_template('create_product.html')
```

HTML
Interfaz de usuario – CRUD URLs

- El **navegador SOLO** efectúa peticiones **GET y POST**
- Debemos contemplar **distintas URLs** para **creación**, **borrado** y **actualización**

Operación	Método HTTP y URL
Read	**GET** /products/<id>
Delete	**POST** /products/<id>/**delete**
Create (**mostrar** formulario)	**GET** /products/**create**
Create (**enviar** datos formulario)	**POST** /products/**create**
Update (**mostrar** formulario)	**GET** /products/<id>/**update**
Update (**enviar** datos formulario)	**POST** /products/<id>/**update**

- Teóricamente, se podría **utilizar GET para borrado, creación y modificación** (usando distintas URLs), pero **se desaconseja por completo** (un *web crawler* efectuaría operaciones de borrado, creación y modificación simplemente por seguir los enlaces de la página)

HTML
Formularios – Atributos de `<form>`

- ***action***: indica la URL a la que se envían los datos del formulario
- ***method***: get / post

- ***target***: indica dónde mostrar la respuesta recibida
  - *_self*: en la misma pestaña, *_blank*: pestaña nueva
  - *_parent* (*frame* padre), *_top* (cuerpo principal página), *framename* (*frame* concreto)

- ***enctype*** (="multipart/form-data" para envío de ficheros)

- ***autocomplete***: on / off
- ***novalidate***: true / false

HTML
Formularios – POST Vs GET

- **GET**
  - Envía los datos en la **URL** como **pares clave – valor**
  - **NUNCA** se debe usar GET para enviar **datos sensibles** (datos visibles en la URL)
  - La longitud de la URL está limitada a 2048 caracteres
  - Permite guardar en los **marcadores del navegador** la URL con los datos del formulario → útil para guardar parámetros de filtrado, búsqueda, etc.

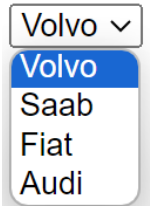
- **POST**
  - Envía los datos en el **cuerpo** de la **petición HTTP** (no se muestran en la URL)
  - No hay limitación del tamaño de los datos
  - No se pueden guardar los datos del formulario en los marcadores del navegador

HTML
Formularios – Elementos

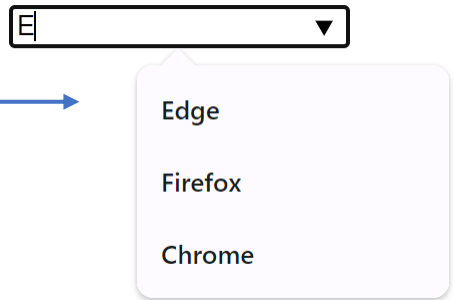
- **<input** *type=" ">**

- checkbox
- color
- date
- datetime-local
- email
- file
- hidden
- image
- month
- number
- password
- radio
- range
- reset
- search
- submit
- tel
- text
- time
- url
- week

- **<label>**
- **<select>**

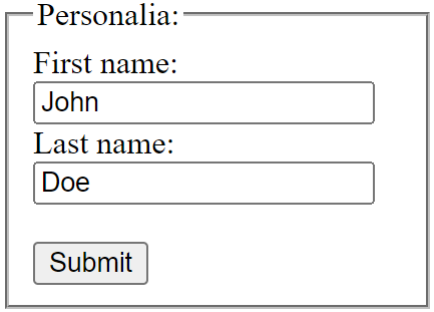


- **<datalist>**



- **<textarea>**

- **<option>, <optiongroup>**
- **<fieldset>, <legend>**



HTML
Formularios – Atributos de `<input>`

- **Generales**
  - *value*
  - *readonly*
  - *disabled*
  - *autofocus, autocomplete*
  - *width, height* [image]
  - *size* (núm. de caracteres visibles) [text, search, url, tel, email, password]
- **Validación**
  - *maxlength*
  - *min, max, step* [number, range, date, datetime-local, month, time, week]
  - *multiple* [email, file]
  - *pattern* = "*regular-expression*" [text, search, url, tel, email, password, date]
  - *placeholder* [text, search, url, tel, email, password]
  - *required*

HTML
Formularios – Atributos de `<input>`

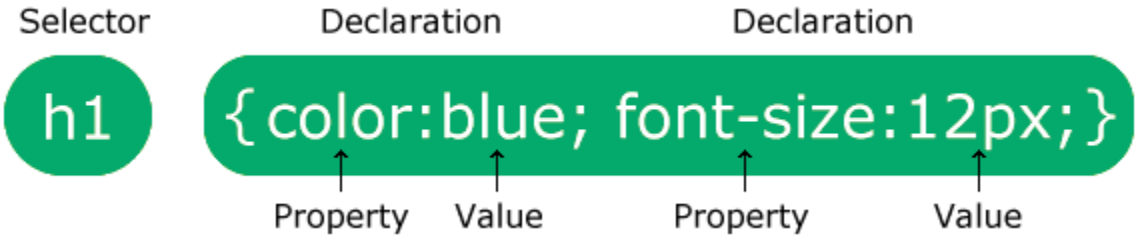
- **Referencia al formulario**
  - *form*: indica el formulario al que pertenece este `<input>`
- **Sobreescritura de atributos del formulario para este `<input>`**
  - *formaction* [submit, image]
  - *formenctype* [submit, image]
  - *formmethod* [submit, image]
  - *formtarget* [submit, image]
  - *formnovalidate* [submit]

CSS
*(**C**ascading **S**tyle **S**heets)*

HTML, CSS & JS

CSS
Intro

- **CSS** permite definir la **forma de presentar** una página HTML
- **Sintaxis**



- **Selector**: selecciona elementos HTML en función de
  - **Simple**: su nombre, *id*, *class*
  - De **combinación**: la relación entre ellos
  - De **pseudo-clase**: su estado
  - De **pseudo-elemento**: selecciona y estiliza una parte de un elemento HTML
  - De **atributo**: un atributo o el valor de un atributo

CSS
Selectores simples y de combinación

- **Simples**

Selector	Ejemplo	Selecciona elemento(s)
#id	`#firstname`	Con *id* = "firstname"
.class	`.intro`	Con *class* = "intro"
element.class	`p.intro`	<p> con *class* = "intro"
*	`*`	Todos
element	`p`	<p>
element,element,..	`div, p`	y <p>

- **De combinación**

Selector	Ejemplo	Selecciona elemento(s)
element element	`div p`	<p> dentro de elementos
element>element	`div > p`	<p> cuyo padre es un elemento
element+element	`div + p`	Primer elemento <p> situado tras elementos
element1~element2	`p ~ ul`	<ul> precedidos de un elemento <p>

CSS
Selectores de pseudo-clase

Selector	Example	Selecciona elemento(s)
:active	`a:active`	<a> con estado *active*
:checked	`input:checked`	<input> con estado *checked*
:disabled	`input:disabled`	<input> con estado *disabled*
:empty	`p:empty`	<p> sin hijos
:enabled	`input:enabled`	<input> con estado *enabled*
:first-child	`p:first-child`	<p> que son el primer hijo de su padre
:first-of-type	`p:first-of-type`	<p> que son el primer elemento <p> su padre
:focus	`input:focus`	<input> que tiene el *focus*
:hover	`a:hover`	<a> que tiene el ratón por encima
:in-range	`input:in-range`	<input> con un valor en ese rango
:invalid	`input:invalid`	<input> con un valor inválido
:lang(language)	`p:lang(it)`	<p> con un valor de atributo *lang* que empieza por "it"
:last-child	`p:last-child`	<p> que son el último hijo de su padre
:last-of-type	`p:last-of-type`	<p> que son el último elemento <p> de su padre
:link	`a:link`	<a> no visitados
:not(selector)	`:not(p)`	Que no son <p>

CSS
Selectores de pseudo-clase (II)

Selector	Example	Selecciona elemento(s)
:nth-child(n)	`p:nth-child(2)`	<p> que son el segundo hijo de su padre
:nth-last-child(n)	`p:nth-last-child(2)`	<p> que son el segundo hijo de su padre, contando desde el último hijo
:nth-last-of-type(n)	`p:nth-last-of-type(2)`	<p> que son el segundo elemento <p> de su padre, contando desde el último hijo
:nth-of-type(n)	`p:nth-of-type(2)`	<p> que son el segundo elemento <p> de su padre
:only-of-type	`p:only-of-type`	<p> que son el único elemento <p>
:only-child	`p:only-child`	<p> que son el único hijo de su padre
:optional	`input:optional`	<input> con ningún atributo *required*
:out-of-range	`input:out-of-range`	<input> con un valor fuera de rango
:read-only	`input:read-only`	<input> con un atributo *readonly*
:read-write	`input:read-write`	<input> con ningún atributo *readonly*
:required	`input:required`	<input> con un atributo *required*
:root	`root`	Raíz del documento
:target	`#news:target`	Se pinchó en una URL apuntando al elemento con *id* = "news"
:valid	`input:valid`	<input> con un valor válido
:visited	`a:visited`	<a> visitados

CSS
Selectores de pseudo-elemento

Selector	Ejemplo	Descripción
::after	`p::after`	Inserta algo después del contenido de cada elemento <p>
::before	`p::before`	Inserta algo antes del contenido de cada elemento <p>
::first-letter	`p::first-letter`	Selecciona la primera letra de cada elemento <p>
::first-line	`p::first-line`	Selecciona la primera línea de cada elemento <p>
::marker	`::marker`	Selecciona los marcadores de una lista de ítems
::selection	`p::selection`	Selecciona la porción de un elemento que ha sido seleccionada por el usuario

CSS
Selectores de atributo

Selector	Ejemplo	Selecciona elemento(s)
[attribute]	`[target]`	Con un atributo *target*
[attribute=value]	`[target="_blank"]`	Con un atributo *target* = "_blank"
[attribute~=value]	`[title~="flower"]`	Con un atributo *title* que contiene una lista de palabras separadas por espacios, una de las cuales es "flower"
[attribute|=value]	`[lang|="en"]`	Con un atributo *lang* que es exactamente "en" o empieza con "en" seguido de un guion
[attribute^=value]	`a[href^="https"]`	<a> con un atributo *href* que empieza por "https"
[attribute$=value]	`a[href$=".pdf"]`	<a> con un atributo *href* que termina en ".pdf"
[attribute*=value]	`a[href*="w3schools"]`	<a> con un atributo *href* que contiene la subcadena "w3schools"

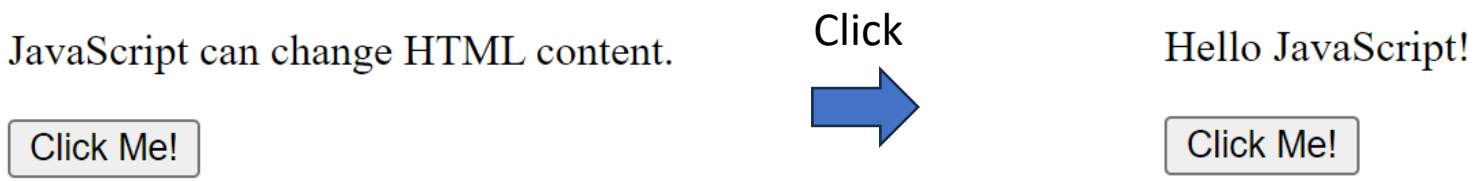
JS
(JavaScript)

HTML, CSS & JS

JS
Intro

- **JS** lenguaje de programación que interpreta el navegador
- **Sintaxis parecida a Python**
  - Llaves `{ }` en lugar de indentación para marcar los bloques
  - Sentencias finalizadas con `;`
  - `function` en lugar de `def`
  - …
- Permite modificar dinám. el **contenido** de un elemento HTML

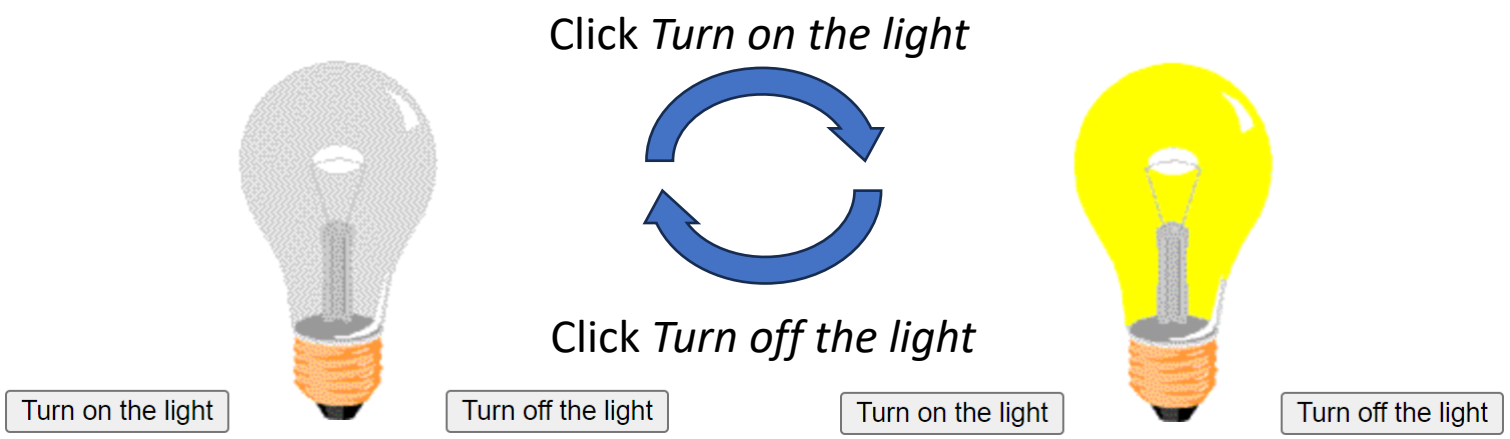
```
<p id="demo">JavaScript can change HTML content.</p>

<button type="button" onclick='document.getElementById("demo").innerHTML =
"Hello JavaScript!"'>Click Me!</button>
```



JS
Intro (II)

- Permite modificar dinám. un **atributo** de un elemento HTML

```
<button onclick="document.getElementById('myImage').src='pic_bulbon.gif'">Turn on
the light</button>

<img id="myImage" src="pic_bulboff.gif" style="width:100px">

<button onclick="document.getElementById('myImage').src='pic_bulboff.gif'">Turn
off the light</button>
```



JS
Intro (III)

- Permite modificar dinámicamente el **estilo** de un elemento HTML

```
<p id="demo">JavaScript can change the style of an HTML element.</p>

<button type="button"
onclick="document.getElementById('demo').style.fontSize='35px'">Click Me!</button>
```

JavaScript can change the style of an HTML element.

Click Me!

Click

JavaScript can change the style of an HTML element.

Click Me!

JS Intro

- Permite **mostrar y ocultar** dinámicamente un elemento HTML

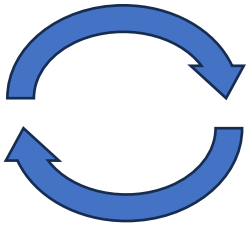
```
<button type="button"
onclick="document.getElementById('demo').style.display='block'">Show</button>

<p id="demo">JavaScript can hide HTML elements.</p>

<button type="button"
onclick="document.getElementById('demo').style.display='none'">Hide</button>
```

Show

JavaScript can hide HTML elements.

Hide

Click *Hide*



Click *Show*

Show Hide

JS
Incluir código JS

- Directamente como manejador de eventos

```
<button onclick="document.getElementById("demo").innerHTML =
"My First JavaScript";">Click me!</button>
```

- Mediante `<script>`

```
<script>
document.getElementById("demo").innerHTML = "My First JavaScript";
</script>
```

  - Puede incluirse tanto en <head> como en <body>
  - El atributo *src* permite indicar que el código JS está en un fichero externo

```
<script src="myScript.js"></script>
```

  - El fichero también puede ser una URL

```
<script src="https://www.w3schools.com/js/myScript.js"></script>
```

JS
Mostrar texto

- Escribiendo en un elemento HTML

```
<p>My First Paragraph.</p>
<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = 5 + 6;
</script>
```

My First Paragraph.

11

- Escribiendo como salida HTML: *document.write()*
  - Reemplaza por completo la página HTML

- Mediante una caja de alerta: *window.alert()*

- En la consola del navegador: *console.log()*

JS
Constantes y variables

- Declaración de una variable
  - x = 5; (solo define una nueva variable si no estaba definida en algún bloque superior)
  - var x = 5; (alcance de bloque)
  - **let x = 5; (alcance de bloque)**
- Declaración de una constante
  - **const x = 5; (alcance de bloque)**

- **Declarar por defecto constantes**. Solo si al final comprobamos que debe ser variable, declararla como tal

JS
Tipos de datos

- **8** tipos de datos
  1. *String*
  2. *Number* (punto flotante 64 bits)
  3. *Bigint* (entero > $2^{53} - 1$)
  4. *Boolean*
  5. *Undefined*
  6. *Null*
  7. *Symbol*
  8. *Object*
     - Un objeto
     - Un *array*
     - Una fecha

JS
Eventos

- `onchange`
- `onclick`
- `onmouseover`
- `onmouseout`
- `onkeydown`
- `onload`
- …

JS
Bucles

- `while`
- `do/while`
- `for`
- **`for/in`**: itera sobre las **propiedades** de un **objeto**

```
const person = {fname:"John", lname:"Doe", age:25};

for (let x in person) {
  console.log(person[x]);
}
```

Consola

```
John
Doe
25
```

- **`for/of`**: itera sobre los **valores** de un objeto **iterable**

```
const cars = ["BMW", "Volvo", "Mini"];

for (let x of cars) {
  console.log(x)
}
```

Consola

```
BMW
Volvo
Mini
```

JS
Validación de entradas de formulario

- Además de las reglas de validación proporcionadas por HTML, JS nos permite implementar nuestras propias reglas de validación
- Ejemplo

HTML

```
<form name="myForm" action="/action_page" onsubmit="return validateForm()"
method="post">
  <label for="name">Name:</label>
  <input type="text" id="name" name="fname">
  <input type="submit" value="Submit">
</form>
```

JS

```
function validateForm() {
  let x = document.forms["myForm"]["fname"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
```

JS
Validación de entradas de formulario (II)

- Ejemplo: Validación de **un solo campo** mediante **`checkValidity()`**

HTML

```
<input id="id1" type="number" min="100" max="300" required>
<button onclick="myFunction()">OK</button>

<p id="demo"></p>
```

OK

JS

```
function myFunction() {
  const inpObj = document.getElementById("id1");
  if (!inpObj.checkValidity()) {
    document.getElementById("demo").innerHTML = inpObj.validationMessage;
  } else {
    document.getElementById("demo").innerHTML = "Input OK";
  }
}
```

50 OK

Value must be greater than or equal to 100.

230 OK

Input OK

400 OK

Value must be less than or equal to 300.

- Mediante **`setCustomValidity()`** podemos establecer la propiedad `validationMessage` del elemento `<input>`

JS
Validación de entradas de formulario (III)

- Propiedades de validación de un `<input>`

Propiedad	Descripción
`validity`	Contiene propiedades booleanas con respecto a la validación de un <input>
`validationMessage`	Contiene el mensaje que el navegador mostrará si la validación falla
`willValidate`	Indica si un elemento <input> será validado o no

- Propiedades de la propiedad `validity` de un `<input>`

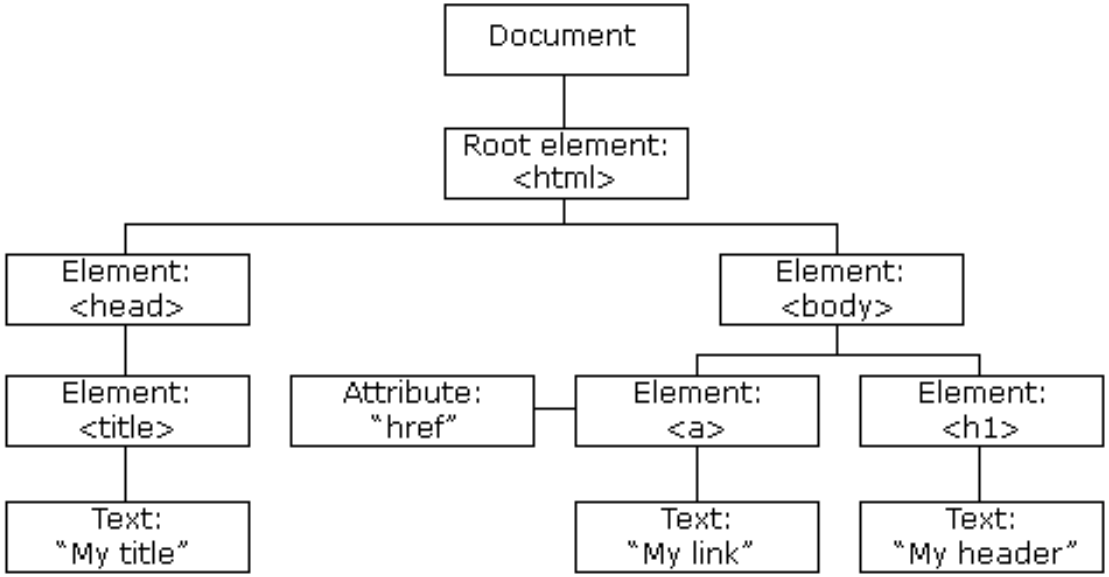
Propiedad	Verdadero si
`validity.customError`	Se ha fijado un mensaje de validación personalizado
`validity.patternMismatch`	El valor no encaja con su atributo *pattern*
`validity.rangeOverflow`	El valor es mayor que su atributo *max*
`validity.rangeUnderflow`	El valor es menor que su atributo *min*
`validity.stepMismatch`	El valor es inválido de acuerdo a su atributo *step*
`validity.tooLong`	El valor excede su atributo *maxLength*
`validity.typeMismatch`	El valor es inválido de acuerdo a su atributo *type*
`validity.valueMissing`	No hay valor y el `<input>` tiene un atributo *required*
`validity.valid`	El valor es válido

El DOM
*(**D**ocument **O**bject **M**odel)*

HTML, CSS & JS

El DOM
Intro

- DOM: estructura de datos que almacena el documento HTML



- JS nos permite acceder programáticamente al DOM
  - Añadir / modificar / borrar elementos del documento HTML
  - Añadir / modificar / borrar atributos de elementos del doc. HTML
  - Cambiar los estilos CSS aplicados al documento HTML

El DOM
Métodos y propiedades

- El DOM define los **elementos del documento HTML como objetos (nodos)**
  - **Métodos**: acciones que podemos realizar sobre ese nodo
  - **Propiedades**: valores de ese nodo que podemos leer / cambiar

- Ejemplo

```
document.getElementById("demo").innerHTML = "Hello World!";
```

  - **getElementById**() es un método del nodo *document* (el DOM del documento HTML) que nos devuelve el nodo del DOM que representa el elemento HTML con atributo *id* = "demo"
  - **innerHTML** es una propiedad de ese nodo

El DOM
Métodos y propiedades principales

- Búsqueda

Método	Encuentra elemento(s)
`document.getElementById(`*id*`)`	Con un atributo *id* determinado
`document.getElementsByTagName(`*name*`)`	Con una etiqueta HTML determinada
`document.getElementsByClassName(`*name*`)`	De una determinada clase
`document.querySelectorAll(`*selector*`)`	Que encajan con un selector CSS determinado

- Modificación

Propiedad	Descripción
element`.innerHTML =` *new html content*	Cambia el contenido del elemento HTML
node`.`*attribute* `=` *new value*	Cambia el valor de un atributo de ese nodo
element`.style.`*property* `=` *new style*	Cambia el estilo del elemento HTML
Método	**Descripción**
node`.setAttribute(`*attribute*`,` *value*`)`	Cambia el valor de un atributo del nodo

El DOM
Métodos y propiedades principales (II)

- Añadir / Eliminar nodos del DOM

Método	Descripción
`document.createElement(element)`	Crea un nodo tipo elemento HTML
`document.createTextNode(text)`	Crea un nodo tipo texto
`document.removeChild(node)`	Elimina un nodo del DOM
`document.appendChild(node)`	Añade un nodo al DOM
`document.replaceChild(new, old)`	Reemplaza un nodo del DOM
`document.write(text)`	Escribe en el flujo HTML de salida

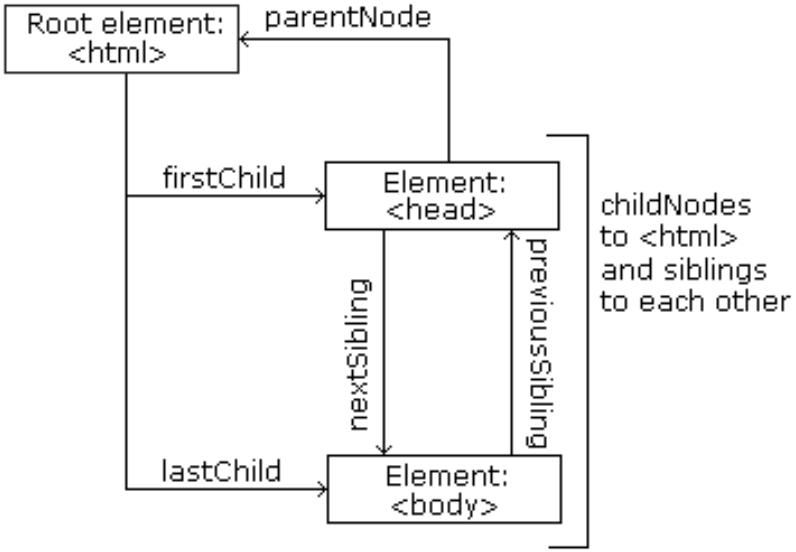
- Añadir manejadores de eventos

Método	Descripción
`document.getElementById(id).onclick = function(){code}`	Añade un manejador de eventos al evento *onclick* para ese elemento HTML

El DOM
Métodos y propiedades principales (III)

- Navegación por el DOM

Propiedades	Nodo(s)
node.parentNode	Padre
node.childNodes[*i*]	Hijo número *i*
node.firstChild	Primer hijo
node.lastChild	Último hijo
node.nextSibling	Hermano siguiente
node.previousSibling	Hermano anterior



- Tipos de nodos

Nodo	Tipo	Ejemplo
ELEMENT_NODE	1	<h1 class="heading">W3Schools</h1>
ATTRIBUTE_NODE	2	class = "heading" *(deprecated)*
TEXT_NODE	3	W3Schools
COMMENT_NODE	8	<!-- This is a comment -->
DOCUMENT_NODE	9	El propio documento HTML (el padre de <html>)
DOCUMENT_TYPE_NODE	10	<!Doctype html>

El DOM
Nodos del DOM

- Los **nodos** del DOM pueden ser **elementos HTML**, **texto…**
- **Error** habitual: pensar que los nodos de tipo **elemento HTML contienen texto**. NO, tienen un **hijo que es un nodo de tipo texto**

- Ejemplo `<title id="demo">DOM Tutorial</title>`
  - Tenemos un nodo que es el elemento HTML `<title>`
  - Tiene un hijo: un nodo de tipo texto cuyo valor es `DOM Tutorial`
  - Acceder a la propiedad `innerHTML` del nodo `<title>` es lo mismo que acceder a la propiedad `nodeValue` de su primer hijo

```
document.getElementById("demo").innerHTML
```

⇕

```
document.getElementById("demo").firstChild.nodeValue
```

⇕

```
document.getElementById("demo").childNodes[0].nodeValue
```

El DOM
Nodos del DOM (II)

- Propiedad *node*`.nodeName` (solo lectura)
  - En un nodo de tipo **elemento** es igual al nombre de la **etiqueta HTML**
  - En un nodo de tipo **atributo** es igual al nombre del **atributo**
  - En un nodo de tipo **texto** es igual a **`#text`**
  - En el nodo de tipo ***document*** es igual **`#document`**

- Propiedad *node*`.nodeValue`
  - En un nodo de tipo **elemento** es igual a **`null`**
  - En un nodo de tipo **atributo** es igual al **valor del atributo**
  - En un nodo de tipo **texto** es igual al **propio texto**

- Propiedad *node*`.nodeType` (solo lectura)
  - Devuelve el tipo de nodo (de acuerdo a la tabla anterior)

El DOM
Propiedades de búsqueda de elementos

Propiedad	Descripción
`document.anchors`	Todos los elementos `<a>` que tienen atributo *name*
`document.baseURI`	URI base absoluta del documento
`document.body`	Elemento `<body>`
`document.cookie`	*Cookie* del documento
`document.doctype`	Tipo de documento del documento
`document.documentElement`	Elemento `<html>`
`document.documentMode`	Modo usado por el navegador
`document.documentURI`	URI del documento
`document.domain`	Nombre de dominio del servidor del documento
`document.embeds`	Todos los elementos `<embed>`
`document.forms`	Todos los elementos `<form>`
`document.head`	Elemento `<head>`
`document.images`	Todos los elementos `<img>`
`document.implementation`	Implementación del DOM
`document.inputEncoding`	Codificación de caracteres del documento
`document.lastModified`	Fecha y hora de actualización del documento
`document.links`	Todos los elementos `<area>` y `<a>` que tienen atributo *href*

El DOM
Propiedades de búsqueda de elementos

Propiedad	Descripción
`document.readyState`	Estado de carga del documento
`document.referrer`	URI del referenciante (el documento que enlazó este documento)
`document.scripts`	Todos los elementos `<script>`
`document.strictErrorChecking`	Comprobación de errores activada o no
`document.title`	Devuelve el elemento `<title>`
`document.URL`	URL completa del documento

El DOM
Colecciones de elementos HTML

- Objeto `HTMLCollectionObject`
  - El método `getElementsByTagName()` devuelve un objeto de clase `HTMLCollectionObject`
  - Es similar a una lista *(array)* de elementos HTML
  - Se puede acceder a un elemento de la lista mediante su índice

```
const myCollection = document.getElementsByTagName("p");
for (let i = 0; i < myCollection.length; i++) {
  myCollection[i].style.color = "red";
}
```

  - NO es exactamente un array ya que no tiene métodos como *valueOf*(), *pop*(), *push*() o *join*()

El DOM
Listas de nodos

- Objeto `NodeList`
  - Diversos métodos devuelven un objeto de clase `NodeList`
  - Es similar a una lista *(array)* de nodos del DOM
  - Se puede acceder a un elemento de la lista mediante su índice

```
const myNodelist = document.querySelectorAll("p");
for (let i = 0; i < myNodelist.length; i++) {
  myNodelist[i].style.color = "red";
}
```

  - NO es exactamente un array ya que no tiene métodos como *valueOf*(), *pop*(), *push*() o *join*()

El DOM
Colecciones HTML Vs Listas de nodos

- Acceso
  - `HTMLCollection`: mediante *name*, *id*, índice
  - `NodeList`: mediante índice
- `HTMLCollection`: siempre una lista **viva**
  - Si se añade un elemento `<li>` a una lista en el DOM, la lista en la `HTMLCollection` también cambia
- `NodeList`: lista **estática** (la mayoría de las veces)
  - Si se añade un elemento `<li>` a una lista en el DOM, la lista en la `NodeList` NO cambia
- `getElementsByClassName()`, `getElementsByTagName()` devuelven `HTMLCollection`
- `querySelectorAll()` devuelve una `NodeList` estática
- `childNodes` devuelve una `NodeList` viva

Notificaciones *Push* desde el servidor HTTP

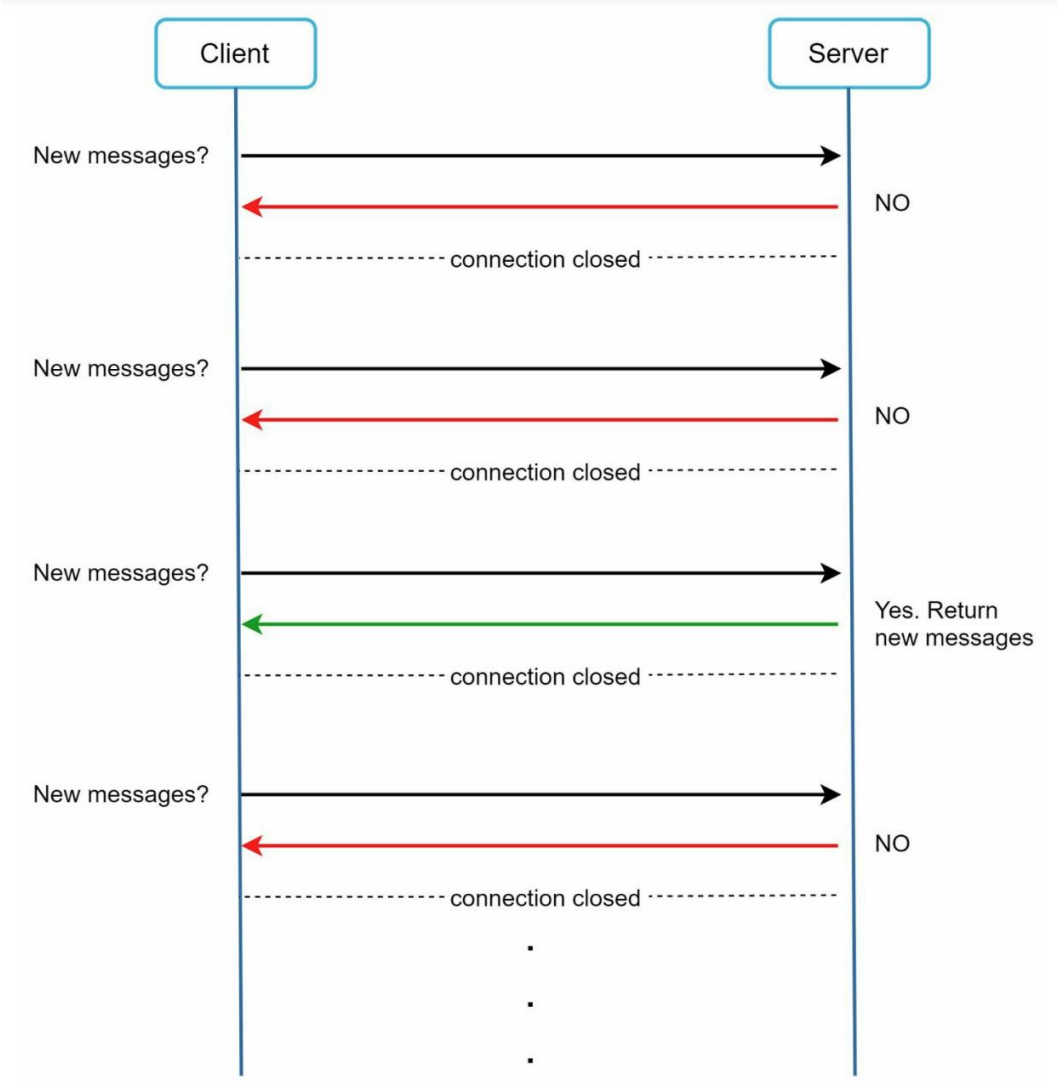
HTML, CSS & JS

Problema

- En HTTP es el cliente quien hace las peticiones para que el servidor le envíe datos
- ¿Cómo puede hacer el servidor para enviarle datos al cliente sin que este se los haya pedido?
- Soluciones
  - *Polling*
  - *Long – Polling*
  - *Websockets*
  - *Server Sent Events* (SSE)

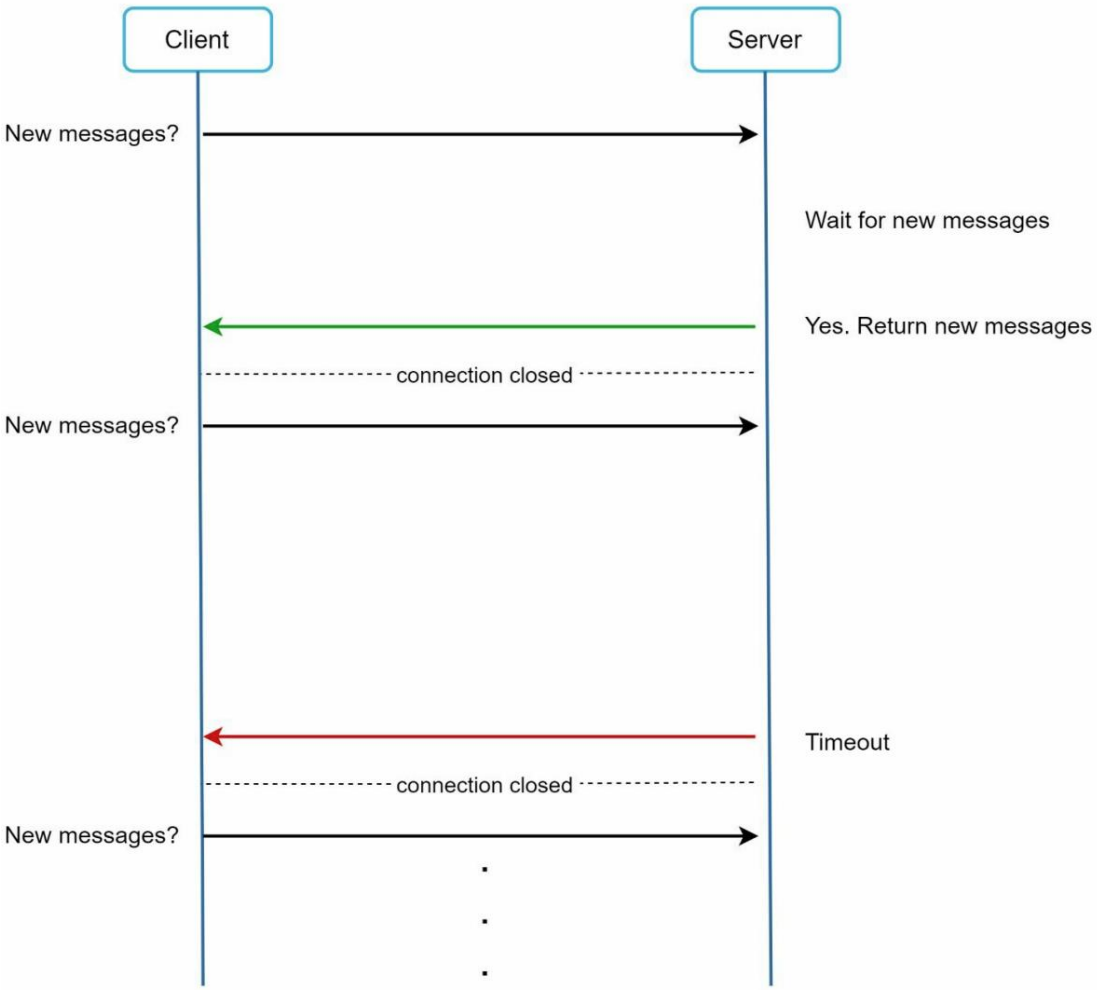
HTTP *Short – Polling*

- **Idea**
  - El cliente pregunta cada poco tiempo al servidor si quiere enviarle algún mensaje
- **Inconvenientes**
  - *Overhead* de abrir y cerrar conexión
  - Exceso de peticiones y respuestas
  - No adecuado para tiempo real



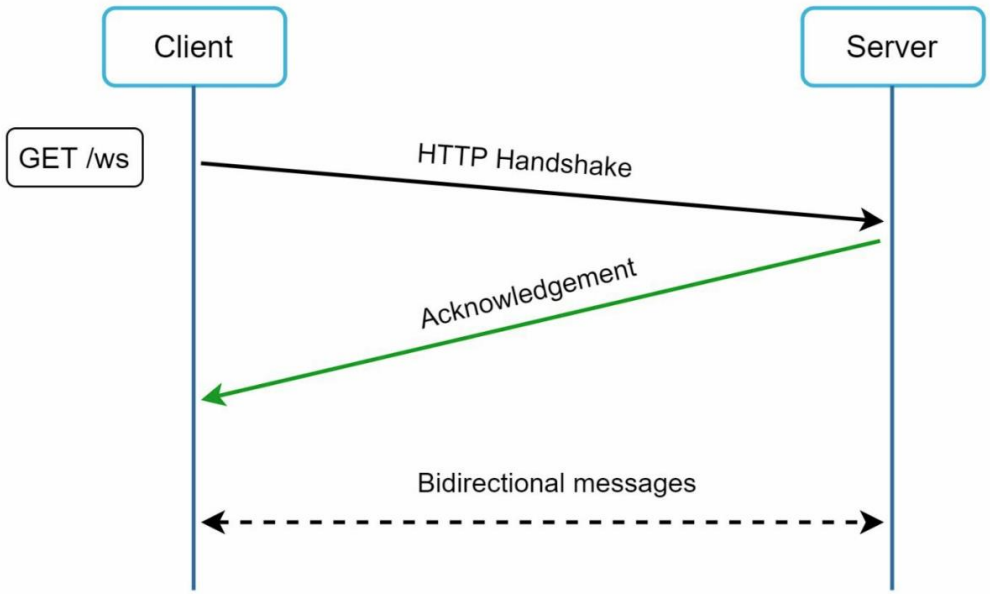
HTTP *Long – Polling*

- **Idea**
  - Cliente pregunta al servidor si quiere enviarle algún mensaje
  - Espera la respuesta del servidor con la conexión abierta
- **Ventajas**
  - Bajo *overhead* de abrir y cerrar conexión
- **Inconvenientes**
  - Exceso de peticiones producidas por *timeouts*



Websockets

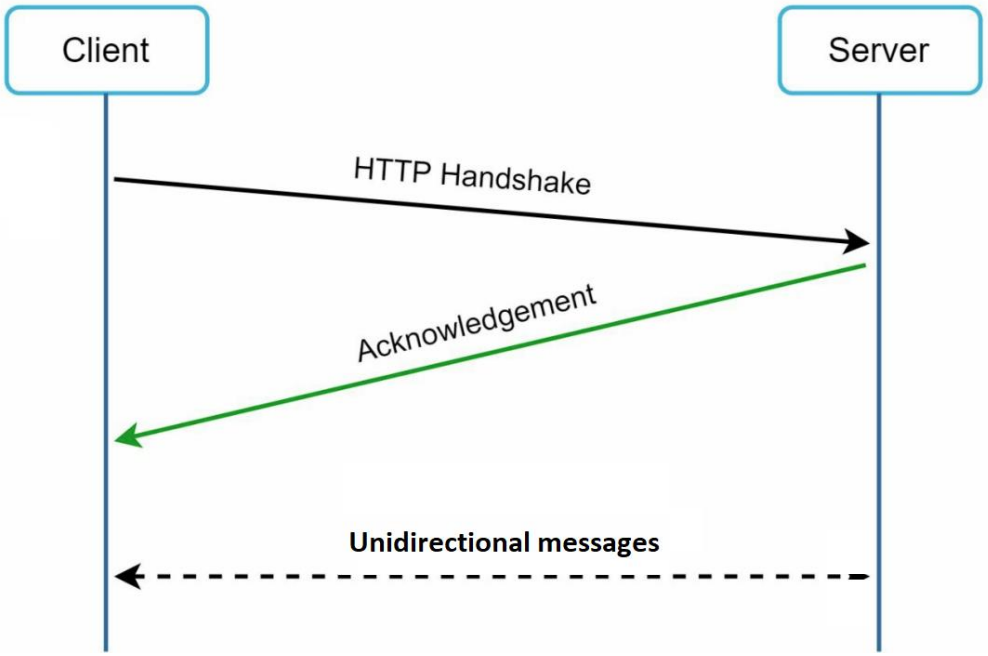
- **Idea**
  - HTTP solo se utiliza para abrir la conexión
  - Una vez abierta la conexión, se convierte en un *socket* **bidireccional**



- **Ventajas**
  - *Overhead* casi nulo ya que el *handshake* para convertir en *socket* la conexión HTTP solo se efectúa al principio

Server Sent Events (SSE)

- **Idea**
  - HTTP solo se utiliza para abrir la conexión
  - Una vez abierta la conexión, se convierte en un *socket* **unidireccional** desde el servidor hacia el cliente



- **Ventajas**
  - *Overhead* casi nulo como ocurría con *websockets*

Socket.IO

- Librería JS que permite **comunicación bidireccional de baja latencia** entre diferentes nodos HTTP
- Se encarga de escoger el mecanismo disponible más adecuado (*websockets* o HTTP *long – polling*)

Flask–Socket.IO

- Librería Python que permite usar **Flask como servidor Socket.IO**
- **Inicialización**

```
from flask import Flask, render_template
from flask_socketio import SocketIO

app = Flask(__name__)
socketio = SocketIO(app)

if __name__ == '__main__':
    socketio.run(app)
```

- La aplicación debe servir una página HTML que cargue la librería JS Socket.IO y establezca la conexión

```
<script src="https://cdn.socket.io/4.7.5/socket.io.min.js"></script>
<script>
    const socket = io();
    socket.on('my event', function(msg) { … });
</script>
```

Flask–Socket.IO – Envío y recepción de mensajes

- Envío Servidor → Cliente
  - ***send()*** (eventos sin nombre)
  - ***emit()*** (eventos con nombre)

- Recepción Cliente
  - ***on()***

- Ejemplo

```
socketio.send('my unnamed event')


json_msg = { "id": "324", "value": 50}

socketio.send(json_msg, json=True)

socketio.emit('my event', json_msg)
```

- Ejemplo

```
</script>
const socket = io();

socket.on('my event', function(msg) {
    console.log(
        "[" +
        new Date().toLocaleString() +
        "] " +
        "My Event received: " +
        JSON.stringify(msg)));
    });
</script>
```